In [1]:

```
import nltk
from nltk.tokenize import sent_tokenize, word_tokenize

nltk.download('punkt')
nltk.download('punkt_tab')

def tokenize(text):
    sentences = sent_tokenize(text)
    words = word_tokenize(text)

    print("Sentences:")
    for i, s in enumerate(sentences, 1):
        print(i, s)

    print("\nWords:")
    for i, w in enumerate(words, 1):
        print(i, w)

text = "NLP is interesting. It helps computers understand language. NLTK is ea

tokenize(text)
```

```
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\sinchana\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\sinchana\AppData\Roaming\nltk_data...

Sentences:
1 NLP is interesting.
2 It helps computers understand language.
3 NLTK is easy to use.

Words:
1 NLP
2 is
3 interesting
4 .
5 It
6 helps
7 computers
8 understand
9 language
10 .
11 NLTK
12 is
13 easy
14 to
15 use
16 .

[nltk_data]   Package punkt_tab is already up-to-date!
```

In [2]:
```python
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords

nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')

def remove_stopwords(text):
    stop_words = set(stopwords.words('english'))
    words = word_tokenize(text)

    filtered = [w for w in words if w.lower() not in stop_words]

    print("Original:", words)
    print("Filtered:", filtered)
    print("Before:", len(words))
    print("After :", len(filtered))
    print("Removed:", len(words) - len(filtered))

text = "This is a simple example of natural language processing."

remove_stopwords(text)
```

```
Original: ['This', 'is', 'a', 'simple', 'example', 'of', 'natural', 'languag
e', 'processing', '.']
Filtered: ['simple', 'example', 'natural', 'language', 'processing', '.']
Before: 10
After : 6
Removed: 4

[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\sinchana\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\sinchana\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\sinchana\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

In [3]:
```python
import nltk
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize

nltk.download('punkt')
nltk.download('punkt_tab')

def stem_text(text):
    stemmer = PorterStemmer()
    words = word_tokenize(text)

    stemmed = [stemmer.stem(w) for w in words]

    print("Original:", text)
    print("Stemmed :", " ".join(stemmed))

text = "The students are studying and building programs."

stem_text(text)
```

```
Original: The students are studying and building programs.
Stemmed : the student are studi and build program .

[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\sinchana\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\sinchana\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

In [5]:
```python
#2a
import nltk
from nltk.tokenize import word_tokenize
from nltk import pos_tag

nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger_eng')

def pos_tagging(text):
    words = word_tokenize(text)
    tagged = pos_tag(words)          # gives (word, tag) pairs

    for word, tag in tagged:
        print(word, '->', tag)

text = 'The average student learned quickly.'
pos_tagging(text)
```

```
The -> DT
average -> JJ
student -> NN
learned -> VBD
quickly -> RB
. -> .

[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\sinchana\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\sinchana\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     C:\Users\sinchana\AppData\Roaming\nltk_data...
[nltk_data]   Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data]       date!
```

In [19]:
```python
#2b
import nltk
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer

nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('wordnet')

def lemmatize_text(text):
    lemmatizer = WordNetLemmatizer()
    words = word_tokenize(text)

    lemmas = [lemmatizer.lemmatize(w, 'n') for w in words]

    print("Original:", text)
    print("Lemmatized:", " ".join(lemmas))

text = "Cats are chasing mice."

lemmatize_text(text)
```

```
Original: Cats are chasing mice.
Lemmatized: Cats are chasing mouse .

[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\sinchana\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\sinchana\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data]     C:\Users\sinchana\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
```

In [9]:
```python
#3a
import nltk
from nltk.tokenize import word_tokenize
from nltk import pos_tag
from nltk.chunk import RegexpParser

nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger_eng')

def chunk_text(text):
    words = word_tokenize(text)
    tagged = pos_tag(words)

    grammar = r"""
       NP: {<DT>?<JJ>*<NN.*>+}
       PP: {<IN><NP>}
       VP: {<VB.*><NP|PP>*}
       CLAUSE: {<NP><VP>}
    """

    parser = RegexpParser(grammar)
    tree = parser.parse(tagged)

    print(tree)

text = "The smart student reads the book in the library."

chunk_text(text)
```

```
(S
  (CLAUSE
    (NP The/DT smart/JJ student/NN)
    (VP
      reads/VBZ
      (NP the/DT book/NN)
      (PP in/IN (NP the/DT library/NN))))
  ./.)
```

```
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\sinchana\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\sinchana\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     C:\Users\sinchana\AppData\Roaming\nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]       date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     C:\Users\sinchana\AppData\Roaming\nltk_data...
[nltk_data]   Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data]       date!
```

In [20]:
```python
#3b
import nltk
from nltk.tokenize import word_tokenize
from nltk import pos_tag
from nltk.chunk import ne_chunk

# Download required resources
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('maxent_ne_chunker_tab')
nltk.download('words')

# Function for Named Entity Recognition
def named_entity(text):
    words = word_tokenize(text)
    tagged = pos_tag(words)
    tree = ne_chunk(tagged)

    print(tree)

# Input sentence
text = "Narendra Modi visited the city of Bengaluru in India."

# Function call
named_entity(text)
```

```
(S
  (PERSON Narendra/NNP)
  (PERSON Modi/NNP)
  visited/VBD
  the/DT
  city/NN
  of/IN
  (GPE Bengaluru/NNP)
  in/IN
  (GPE India/NNP)
  ./.)

[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\sinchana\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\sinchana\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     C:\Users\sinchana\AppData\Roaming\nltk_data...
[nltk_data]   Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data]       date!
[nltk_data] Downloading package maxent_ne_chunker_tab to
[nltk_data]     C:\Users\sinchana\AppData\Roaming\nltk_data...
[nltk_data]   Package maxent_ne_chunker_tab is already up-to-date!
[nltk_data] Downloading package words to
[nltk_data]     C:\Users\sinchana\AppData\Roaming\nltk_data...
[nltk_data]   Package words is already up-to-date!
```

In [31]:
```python
#4a

from sklearn.feature_extraction.text import TfidfVectorizer

docs = [
    "natural language processing is interesting",
    "natural language processing uses python",
    "python is useful for language processing"
]

vectorizer = TfidfVectorizer(stop_words='english')
matrix = vectorizer.fit_transform(docs)

print("TF-IDF Matrix:")
print(matrix.toarray())

print("Vocabulary:")
print(vectorizer.get_feature_names_out())
```

```
TF-IDF Matrix:
[[0.66283998 0.39148397 0.50410689 0.39148397 0.         0.
  0.        ]
 [0.         0.34957775 0.45014501 0.34957775 0.45014501 0.
  0.59188659]
 [0.         0.39148397 0.         0.39148397 0.50410689 0.66283998
  0.        ]]
Vocabulary:
['interesting' 'language' 'natural' 'processing' 'python' 'useful' 'uses']
```

In [30]:
```python
#4b
import nltk

grammar = nltk.CFG.fromstring("""
S -> NP VP
NP -> 'your'
VP -> 'work'
""")

parser = nltk.ChartParser(grammar)

sentence = "your work"
tokens = sentence.split()

trees = list(parser.parse(tokens))

if trees:
    print("Sentence is grammatically valid.")
    for tree in trees:
        print(tree)
        tree.draw()
else:
    print("Sentence is not valid.")
```

```
Sentence is grammatically valid.
(S (NP your) (VP work))
```

In [32]:
```
#5a
import nltk
from nltk.util import ngrams

nltk.download('punkt')
nltk.download('punkt_tab')

text = "I love natural language processing"

words = nltk.word_tokenize(text)

for n in range(1, 4):
    print(n, list(ngrams(words, n)))
```

```
1 [('I',), ('love',), ('natural',), ('language',), ('processing',)]
2 [('I', 'love'), ('love', 'natural'), ('natural', 'language'), ('language',
'processing')]
3 [('I', 'love', 'natural'), ('love', 'natural', 'language'), ('natural', 'l
anguage', 'processing')]

[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\sinchana\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\sinchana\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

In [33]:
```
#5b

from collections import Counter

corpus = ["this is my cat", "this is my dog"]

words = " ".join(corpus).split()

uni = Counter(words)
bi = Counter(zip(words, words[1:]))

test = "this is my cat".split()
p = 1

for w1, w2 in zip(test, test[1:]):
    p *= bi[(w1, w2)] / uni[w1] if bi[(w1, w2)] else 0

print("Sentence:", " ".join(test))
print("Probability:", p)
```

```
Sentence: this is my cat
Probability: 0.5
```

In [ ]:

```python
#6
import stanza

stanza.download('en')
nlp = stanza.Pipeline('en', processors='tokenize,ner')

def extract_entities(text):
    doc = nlp(text)

    for e in doc.ents:
        print(e.text, "->", e.type)

documents = [
    "Barack Obama was born in Hawaii.",
    "Microsoft was founded by Bill Gates.",
    "Apple is located in California.",
    "John visited Bengaluru on Monday."
]

for i, text in enumerate(documents, 1):
    print("\nDocument", i)
    extract_entities(text)
```

In [35]:
```
#7
import nltk
from collections import Counter
from nltk.corpus import reuters, stopwords
from sklearn.feature_extraction.text import TfidfVectorizer

nltk.download('reuters')
nltk.download('stopwords')
nltk.download('punkt')
nltk.download('punkt_tab')

def analyze():
    docs = [reuters.raw(f) for f in reuters.fileids()]
    sw = set(stopwords.words('english'))
    count = Counter()

    for d in docs:
        count.update(w for w in nltk.word_tokenize(d.lower()) if w in sw)

    print("Stopwords:", count.most_common(10))

    v = TfidfVectorizer(stop_words='english')
    x = v.fit_transform(docs)
    words = v.get_feature_names_out()
    scores = x[0].toarray()[0]

    print("Top TF-IDF words:")
    for i in scores.argsort()[-10:][::-1]:
        print(words[i], round(scores[i], 3))

analyze()
```

```
[nltk_data] Downloading package reuters to
[nltk_data]     C:\Users\sinchana\AppData\Roaming\nltk_data...
[nltk_data]   Package reuters is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\sinchana\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\sinchana\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\sinchana\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!

Stopwords: [('the', 69245), ('of', 36749), ('to', 36275), ('in', 29217), ('a
nd', 25616), ('a', 24723), ('for', 13420), ('it', 11087), ('on', 9094), ('fr
om', 8217)]
Top TF-IDF words:
japan 0.323
trade 0.321
businessmen 0.171
tariffs 0.169
hong 0.153
kong 0.153
said 0.153
exports 0.152
electronics 0.151
taiwan 0.146
```

In [36]:
```
#8
import nltk
from nltk.corpus import movie_reviews
from sklearn.pipeline import make_pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB

nltk.download('movie_reviews')

X = [movie_reviews.raw(f) for f in movie_reviews.fileids()]
y = [movie_reviews.categories(f)[0] for f in movie_reviews.fileids()]

model = make_pipeline(TfidfVectorizer(stop_words='english'),
                      MultinomialNB())

model.fit(X, y)

for text in ["This movie is excellent", "This movie is terrible"]:
    print(text, ":", model.predict([text])[0].upper())
```

```
[nltk_data] Downloading package movie_reviews to
[nltk_data]     C:\Users\sinchana\AppData\Roaming\nltk_data...
[nltk_data]   Package movie_reviews is already up-to-date!

This movie is excellent : POS
This movie is terrible : NEG
```

In [ ]: